IFMT - Instituto Federal de Mato Grosso

Eng. da Computação



Criptografia de Chave Simétrica:

AES-128 NA PRÁTICA

Prof. Dr. Ruy de Oliveira

AES-128

passo a passo, byte a byte

Do texto claro ao texto cifrado (e de volta), com todos os valores conferidos e o vetor de teste oficial do padrão FIPS-197

32	88	31	e0
43	5a	31	37
f6	30	98	07
a8	8d	a2	34

SUMÁRIO

O que vamos construir, passo a passo

01

O vetor de teste

O texto claro, a chave e a matriz de estado

02

Expansão de chave

Como 16 bytes viram 11 chaves de round

03

Round 1 completo

SubBytes, ShiftRows, MixColumns e AddRoundKey, byte a byte

04

Rounds 2 a 10

A repetição da mecânica até o texto cifrado final

05

Decifragem

Revertendo cada etapa até recuperar o texto original

06

Implementação em Python

O algoritmo completo, do zero, sem bibliotecas

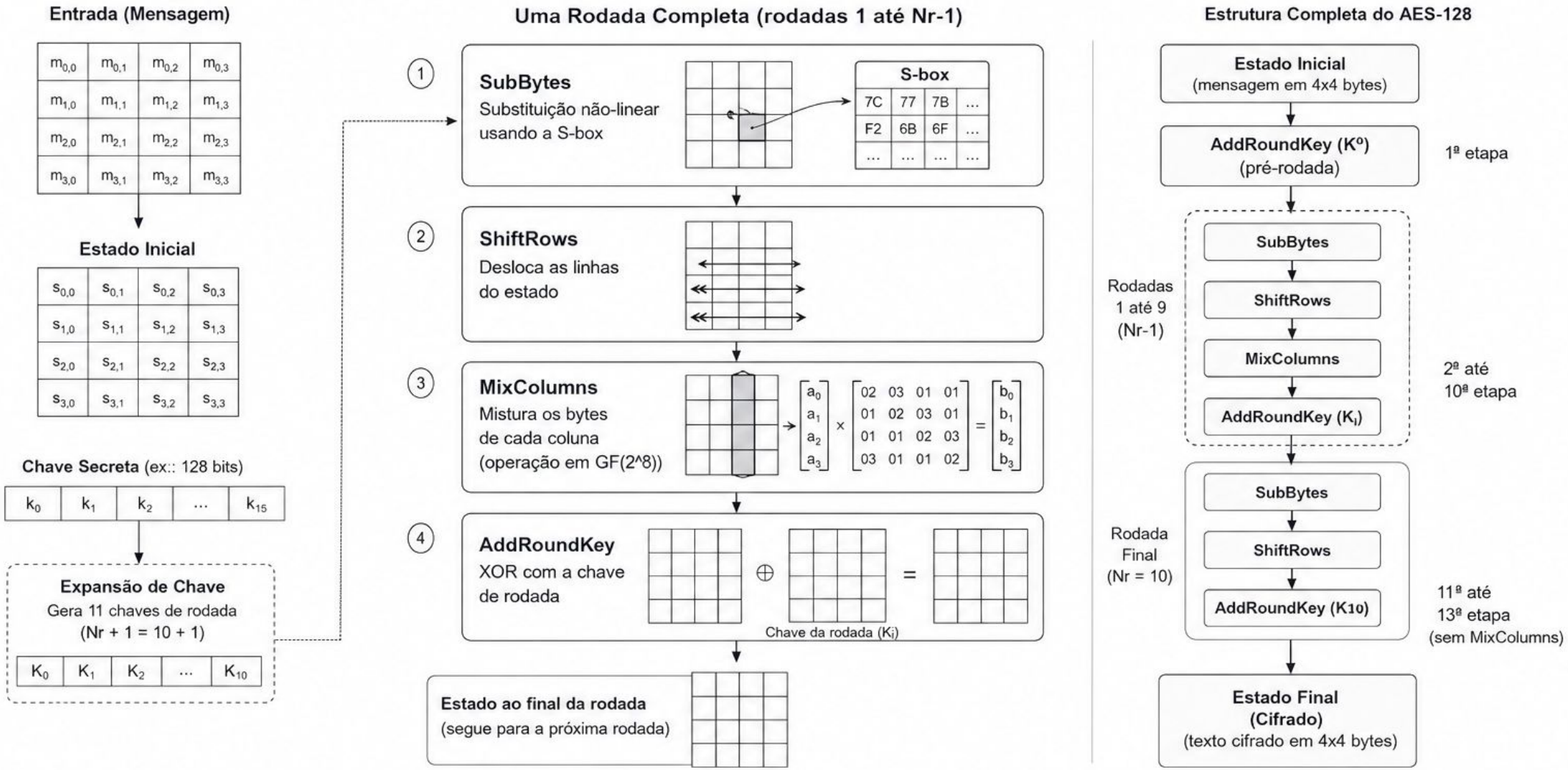
Por o AES use várias mecanismos e rodadas?

- O AES não se baseia na incapacidade de o atacante testar chaves; baseia-se no fato de o espaço de chaves ser tão grande que uma busca exaustiva se torna computacionalmente inviável (*Força Bruta*)
- Uma cifra forte não deve apenas ter um espaço de chaves grande. Ela também deve impedir que um atacante explore relações matemáticas entre o texto simples, o texto cifrado e a chave
- A S-box rompe relações lineares, impedindo que um atacante simplesmente acompanhe relações algébricas entre entrada, saída e chave
- A segurança do AES não se baseia em manter o algoritmo em segredo. Baseia-se em manter a chave em segredo
- **RESUMO:** Grande espaço de chaves + não linearidade + difusão + múltiplas rodadas + chaves de rodada diferentes → torna a recuperação da chave computacionalmente inviável por métodos conhecidos

Por que é difícil quebrar o AES?

- Grande espaço de chaves
  - O AES-128 possui (2^{128}) chaves possíveis
  - Isso torna um ataque de força bruta computacionalmente inviável
- S-box – não linearidade
  - A S-box introduz não linearidade no algoritmo.
  - Isso dificulta estabelecer relações matemáticas simples entre texto claro, texto cifrado e chave
- ShiftRows e MixColumns – difusão
  - Essas operações espalham as alterações pelo estado do AES
  - Uma pequena alteração no texto claro pode provocar alterações em muitos bytes do texto cifrado
  - Isso contribui para o chamado efeito avalanche
- Múltiplas rodadas
  - O AES repete suas transformações em várias rodadas:
    - AES-128 → 10 rodadas
    - AES-192 → 12 rodadas
    - AES-256 → 14 rodadas
- Cada rodada utiliza uma chave de rodada diferente
- Expansão da chave
- A chave original é expandida para gerar as diferentes round keys
  - A expansão utiliza operações como S-box, RotWord, SubWord e Rcon
- Rcon e S-box na expansão da chave
  - A S-box introduz não linearidade também no processo de geração das chaves
  - O Rcon introduz constantes diferentes em cada etapa, ajudando a evitar padrões e simetrias na expansão da chave
- Ausência de atalho prático conhecido
  - Para uma implementação correta e uma chave adequadamente protegida, não há atualmente um método prático conhecido para recuperar a chave AES de forma eficiente
  - Assim, um atacante que tente quebrar o AES por força bruta enfrenta um espaço de busca extremamente grande

AES – Uma Rodada Completa (Cifração)



Resumo das Transformações

- **SubBytes**: substitui cada byte usando a S-box (não linear)
- **ShiftRows**: desloca ciclicamente as linhas do estado
- **MixColumns**: mistura os bytes de cada coluna (difusão)
- **AddRoundKey**: combina o estado com a chave da rodada (XOR)

Parâmetros AES-128

- Tamanho do bloco: 128 bits (4x4 bytes)
- Tamanho da chave: 128 bits
- Número de rodadas (Nr): 10
- Chaves geradas: 11 (K0 até K10)

Operações

- Todas as operações são em GF(2^8)
- XOR = soma módulo 2
- Multiplicações de MixColumns usam o polinômio redutor: $x^8 + x^4 + x^3 + x + 1$ (0x11B)

Texto claro e chave — o ponto de partida

Por que usar valores oficiais — vamos usar o vetor de teste do próprio padrão AES (FIPS-197, Apêndice B) — assim, cada número calculado aqui pode ser conferido em qualquer biblioteca de criptografia real.

TEXTO CLARO (16 bytes)

```
32 43 f6 a8  88 5a 30 8d  31 31 98 a2  e0 37 07 34
```

CHAVE AES-128 (16 bytes)

```
2b 7e 15 16  28 ae d2 a6  ab f7 15 88  09 cf 4f 3c
```

O que vem a seguir — vamos cifrar este texto claro com esta chave, conferindo cada byte intermediário, até chegar a um texto cifrado de 16 bytes.

A matriz de estado (state)

A convenção do AES — os 16 bytes não são tratados como uma fila — eles preenchem uma matriz 4×4 COLUNA POR COLUNA, não linha por linha.

```
32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34
índices:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```



Estado inicial (4×4)

32	88	31	e0
43	5a	31	37
f6	30	98	07
a8	8d	a2	34

Confira — a primeira COLUNA da matriz (32 43 f6 a8) é exatamente os 4 primeiros bytes do texto claro — é essa unidade (a coluna) que várias operações do AES manipulam.

De 16 bytes a 11 chaves de round

O problema — o AES-128 usa uma chave de round DIFERENTE em cada um dos 10 rounds (mais uma para o round 0) — 11 chaves de 16 bytes, totalizando 176 bytes

A solução — um algoritmo determinístico ("**key schedule**") expande os 16 bytes originais (da chave original AES-128) nessas 11 chaves

→ A chave de 128 bits é dividida em quatro palavras de 32 bits: W0, W1, W2, W3

AS 4 PRIMEIRAS PALAVRAS (DAS CHAVEs) SÃO A PRÓPRIA CHAVE

```
w0 = 2b7e1516      w1 = 28aed2a6      w2 = abf71588      w3 = 09cf4f3c
```

A partir daqui, cada nova palavra w[i] é gerada a partir de w[i-1] e w[i-4]. A cada 4 palavras (início de uma nova chave de round), passos extras entram em ação:



RotWord: gira w3 um byte para a esquerda
SubWord: aplica a S-box a cada um dos 4 bytes
Rcon: constante da rodada

Calculando w4 na prática (parte 1 de 2)

A cada nova palavra (início de uma nova chave de round), **quatro passos extras** entram em ação na expansão:

1. RotWord
Gira a palavra um byte à esquerda.

→

2. SubWord
Aplica a S-box a cada um dos 4 bytes da Word.

→

3. XOR com Rcon
Introduz uma constante dependente da rodada.

→

4. XOR com W_{i-4}
Combina o resultado com a palavra correspondente.

SBOX: Não-Linearidade

Garante a confusão e independência matemática entre as chaves em cada round

Exemplos de Substituição:

SBOX(cf) = 8a
SBOX(3c) = eb
SBOX(4f) = 84
SBOX(09) = 01

https://en.wikipedia.org/wiki/Rijndael_S-box

Rcon (Round Constant)

Introduz uma dependência do round na geração das novas palavras das chaves

Round	Valor Rcon
1	01 00 00 00
2	02 00 00 00
3	04 00 00 00

S-box → **Introduz não linearidade** → Se a expansão de chave fosse formada apenas por XORs e rotações, haveria relações matemáticas muito simples entre as palavras
Rcon → **Introduz assimetria/diferenciação entre as rodadas**

Calculando w4 na prática (parte 2 de 2)

Passo a Passo da Resolução

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Passo 1: RotWord

Gira W_3 (09cf4f3c) um byte à esquerda:

RotWor(W_3) = cf4f3c09

Passo 2: SubWord

Aplica a S-box a cada um dos 4 bytes:

```
SBOX(cf) = 8a
SBOX(3c) = eb
SBOX(4f) = 84
SBOX(09) = 01
```

SubWord(RotWord(W_3)) = 8a84eb01

Passo 3: XOR com Rcon

8a84eb01 ⊕ 01000000 = 8b84eb01

Passo 4: XOR com W_0 (2b7e1516)

8b84eb01 ⊕ 2b7e1516 = a0fafe17

Rcon & SBOX

Rodada	Valor Rcon (Hexadecimal)
1	01 00 00 00
2	02 00 00 00
3	04 00 00 00
4	08 00 00 00
5	10 00 00 00
6	20 00 00 00
7	40 00 00 00
8	80 00 00 00
9	1B 00 00 00
10	36 00 00 00

Visualização Binária do XOR (Passo 4)

```
(8b84eb01) 1000 1011 1000 0100 1110 1011 0000 0001
(2b7e1516) 0010 1011 0111 1110 0001 0101 0001 0110
(a0fafe17) 1010 0000 1111 1010 1111 1110 0001 0111
```

Calculando W_5, W_6 e W_7

Desenvolvimento dos Cálculos Binários (XOR)

```
w0 = 2b7e1516
w1 = 28aed2a6
w2 = abf71588
w3 = 09cf4f3c
w4 = a0fafe17
```

$W_5 = W_4 \oplus W_1$ = a0fafe17 ⊕ 28aed2a6 = 88542cb1

```
(a0fafe17) 1010 0000 1111 1010 1111 1110 0001 0111
(28aed2a6) 0010 1000 1010 1110 1101 0010 1010 0110
(88542cb1) 1000 1000 0101 0100 0010 1100 1011 0001
```

$W_6 = W_5 \oplus W_2$ = 88542cb1 ⊕ abf71588 = 23a33939

```
(88542cb1) 1000 1000 0101 0100 0010 1100 1011 0001
(abf71588) 1010 1011 1111 0111 0001 0101 1000 1000
(23a33939) 0010 0011 1010 0011 0011 1001 0011 1001
```

$W_7 = W_6 \oplus W_3$ = 23a33939 ⊕ 09cf4f3c = 2a6c7605

```
(23a33939) 0010 0011 1010 0011 0011 1001 0011 1001
(09cf4f3c) 0000 1001 1100 1111 0100 1111 0011 1100
(2a6c7605) 0010 1010 0110 1100 0111 0110 0000 0101
```

Regra Simples de Expansão

$$W_i = W_{i-1} \oplus W_{i-4}$$

Usada para completar a chave do Round 1

Composição das Chaves

Primeira Chave: chave[0] = $W_0W_1W_2W_3$

2b7e151628aed2a6**abf71588**09cf4f3c

Segunda Chave: chave[1] = $W_4W_5W_6W_7$

a0fafe1788542cb1**23a33939**2a6c7605

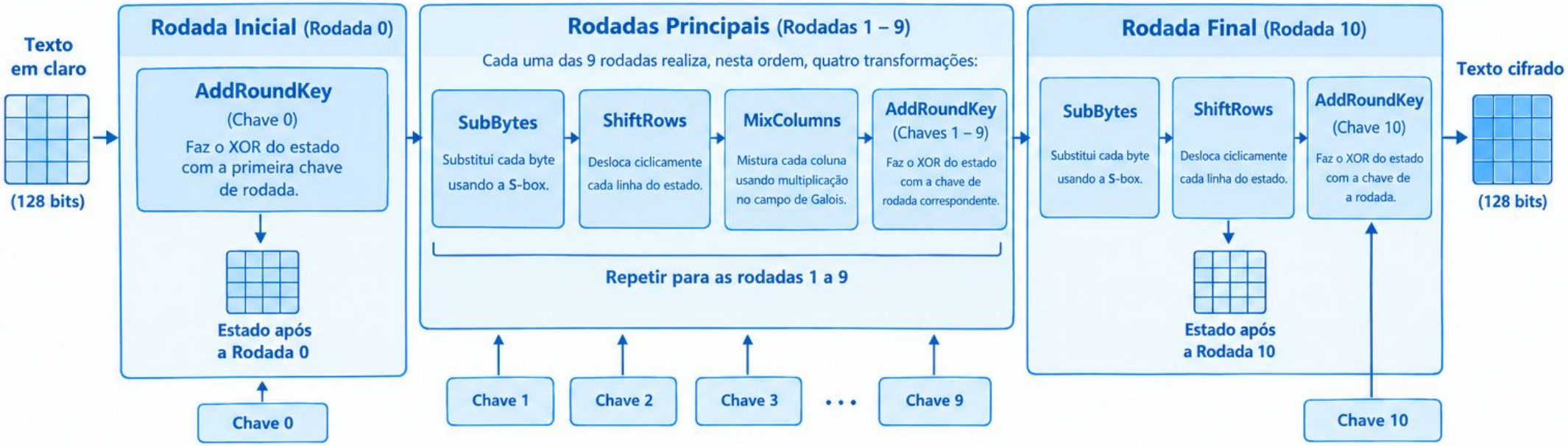
As 11 chaves de round, completas

Repetindo essa lógica 44 vezes (11 rounds × 4 palavras), chegamos à tabela completa:

0	2b7e151628aed2a6abf7158809cf4f3c	*(a própria chave original)*
1	a0fafe1788542cb123a339392a6c7605	
2	f2c295f27a96b9435935807a7359f67f	
3	3d80477d4716fe3e1e237e446d7a883b	
4	ef44a541a8525b7fb671253bdb0bad00	
5	d4d1c6f87c839d87caf2b8bc11f915bc	
6	6d88a37a110b3efddbf98641ca0093fd	
7	4e54f70e5f5fc9f384a64fb24ea6dc4f	
8	ead27321b58dbad2312bf5607f8d292f	
9	ac7766f319fadc2128d12941575c006e	
10	d014f9a8c9ee2589e13f0cc8b6630ca6	

Processo de Cifração do AES Após a Geração das Chaves

Após a geração das 11 chaves de rodada (Key Expansion), o processo de cifração do AES é realizado da seguinte forma:

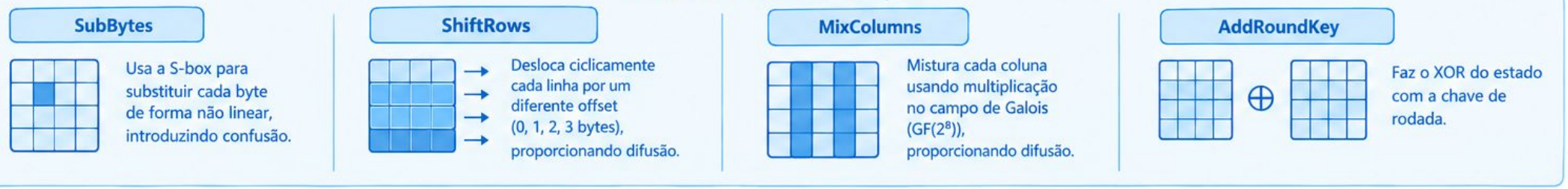


Sequência Resumida



O texto cifrado é o estado resultante após a última AddRoundKey.

As Quatro Transformações



Round 0 — a primeira mistura com a chave

Antes do round 1 — o estado inicial (texto claro) é combinado, byte a byte via XOR, com a Round key 0 — que é a própria chave original.

Estado inicial

32	88	31	e0
43	5a	31	37
f6	30	98	07
a8	8d	a2	34

⊕

2b	28	ab	09
7e	ae	f7	cf
15	d2	15	4f
16	a6	88	3c

=

Fim do round 0

19	a0	9a	e9
3d	f4	c6	f8
e3	e2	8d	48
be	2b	2a	08

CONFERINDO O PRIMEIRO BYTE (destacado acima)

`0x32 ⊕ 0x2b = 0x19` (00110010 XOR 00101011 = 00011001)

Esse XOR simples é repetido para os 16 bytes do estado.

SubBytes — a única etapa não linear

Fundamentação — cada byte é tratado como elemento do corpo finito, ou campo de Galois, GF(2^8), invertido nesse corpo, e depois passa por uma transformação afim fixa. O resultado é uma tabela (S-box) sem nenhuma relação linear simples com a entrada.

Antes

19	a0	9a	e9
3d	f4	c6	f8
e3	e2	8d	48
be	2b	2a	08

Depois (SubBytes)

d4	e0	b8	1e
27	bf	b4	41
11	98	5d	52
ae	f1	e5	30

Sbox

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

EXEMPLO DE LOOKUP NA S-BOX

```
sbox[19] = d4      sbox[a0] = e0      sbox[9a] = b8      sbox[e9] = 1e
```

256 valores fixos, tabelados — não é uma conta feita "na hora" durante a cifragem.

ShiftRows — espalhando bytes entre colunas

Fundamentação — cada linha da matriz gira ciclicamente para a esquerda: linha 0 não muda, linha 1 desloca 1 posição, linha 2 desloca 2, linha 3 desloca 3.

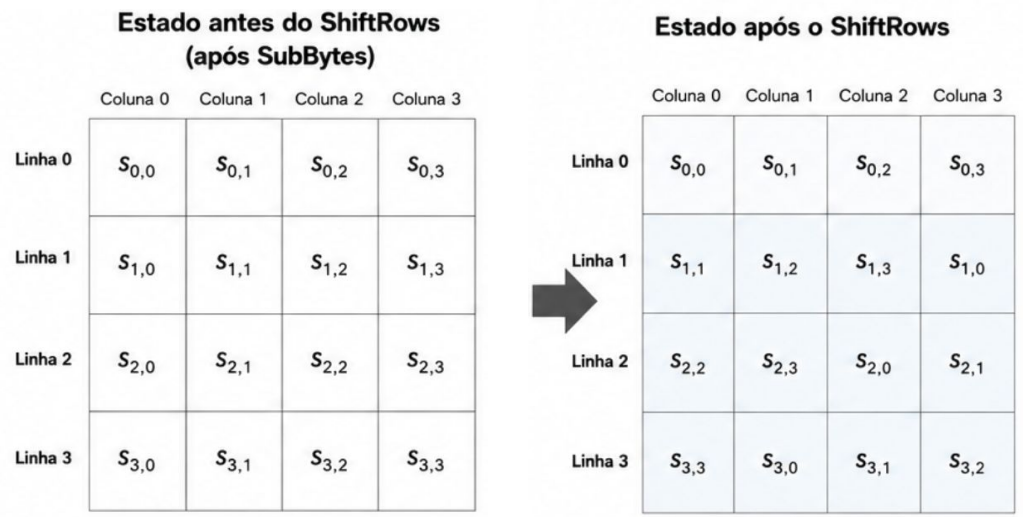
Por quê — SubBytes só mistura cada byte "dentro de si mesmo"; ShiftRows redistribui bytes entre colunas, preparando o terreno para o MixColumns espalhar essa influência de verdade.

Antes

d4	e0	b8	1e
27	**bf**	**b4**	**41**
11	98	5d	52
ae	f1	e5	30

Depois (ShiftRows)

d4	e0	b8	1e
bf	**b4**	**41**	**27**
5d	52	11	98
30	ae	f1	e5



Repare na linha destacada: 27 bf b4 41 → bf b4 41 27 (rotação de 1 posição para a esquerda).

MixColumns — a matemática do AES (parte 1)

O que é — cada coluna do estado é multiplicada por uma matriz fixa — mas os "números" vivem no **corpo finito GF(2^8)**, não nos inteiros normais.

O corpo finito GF(2^8) é uma estrutura algébrica composta por **exatamente 256 elementos**, na qual cada byte é interpretado como um elemento desse corpo. Uma de suas propriedades fundamentais é que todo **elemento não-nulo possui um inverso** multiplicativo único dentro da estrutura.

Depois (ShiftRows)

d4	e0	b8	1e
bf	b4	41	27
5d	52	11	98
30	ae	f1	e5

```
| 02 03 01 01 |
| 01 02 03 01 |
| 01 01 02 03 |
| 03 01 01 02 |
```

matriz fixa de multiplicação (MixColumns)

2 × byte — a operação "xtime"

```
0xd4 em binario  = 11010100
desloca 1 bit    = 10101000
bit mais alto era 1, entao:
XOR 0x1B (reducao)
resultado = 0xb3
```

Multiplicar por 2 (xtime) — desloca o byte 1 bit à esquerda; se "vazou" um bit (byte ≥ 0x80 antes do shift), faz XOR com 0x1B — o polinômio redutor do AES ($x^8+x^4+x^3+x+1$).

Multiplicar por 3 — é simplesmente (2×a) XOR a. Toda a matemática do MixColumns cabe nessas duas regras.

MixColumns — a matemática do AES (Cont. parte 1)

Matriz MixColumns

02	**03**	**01**	**01**
01	02	03	01
01	01	02	03
03	01	01	02

×

Estado (ShiftRows)

d4	e0	b8	1e
bf	b4	41	27
5d	52	11	98
30	ae	f1	e5

=

Resultado (MixColumns)

04	e0	48	28
66	cb	f8	06
81	19	d3	26
e5	9a	7a	4c

Operação de Multiplicação: Cada coluna do novo estado é calculada multiplicando a matriz fixa por uma coluna correspondente do estado atual. No exemplo destacado, a multiplicação da primeira linha de MixColumns pela primeira coluna do estado gera o primeiro byte do resultado:
`04 = (02 · d4) ⊕ (03 · bf) ⊕ (01 · 5d) ⊕ (01 · 30)` (cálculos feitos no corpo finito $GF(2^8)$)

MixColumns — a matemática do AES (parte 2)

Calculando a PRIMEIRA COLUNA do nosso estado (d4 bf 5d 30) após o ShiftRows:

```
byte0 = (2*d4) ^ (3*bf) ^ (5d) ^ (30)
= b3 ^ da ^ 5d ^ 30 = 04

byte1 = (d4) ^ (2*bf) ^ (3*5d) ^ (30)
= d4 ^ 65 ^ e7 ^ 30 = 66

byte2 = (d4) ^ (bf) ^ (2*5d) ^ (3*30)
= d4 ^ bf ^ ba ^ 50 = 81

byte3 = (3*d4) ^ (bf) ^ (5d) ^ (2*30)
= 67 ^ bf ^ 5d ^ 60 = e5
```

```
| 02 03 01 01 |
| 01 02 03 01 |
| 01 01 02 03 |
| 03 01 01 02 |
```

Após ShiftRows

d4	e0	b8	1e
bf	b4	41	27
5d	52	11	98
30	ae	f1	e5

Após MixColumns

04	e0	48	28
66	cb	f8	06
81	19	d3	26
e5	9a	7a	4c

Nova coluna 0 = 04 66 81 e5 (as outras 3 colunas seguem o mesmo processo)

Por que essa matriz específica? ela é um código MDS — garante que mudar 1 byte de entrada muda todos os 4 bytes de saída daquela coluna, gerando o efeito avalanche.

* MDS: Maximum Distance Separable

MixColumns — Cálculo: Primeiro Elemento da Primeira Coluna

(02 . D4) XOR (03 . BF) XOR (02. 5D) XOR (01 . 30) (em FG(2^8))

Multiplicação no corpo finito GF(2^8):
- 02 . byte → desloca à esq. e XOR com 0x1B se MSB =1
- 03 . byte = (02 . byte) XOR byte
- 01 . byte = byte

02 . D4
02 . D4 = 1bit_esq(11010100) = 110101000
10101000
(1B) 00011011
(B3) 10110011
02 . d4 = B3

01 . 5D
01 . 5D = 5D

01 . 30
01 . 30 = 30

03 . BF
02 . BF = 1bit_esq(10111111) = 101111110
01111110
(1B) 00011011
(64) 01100101
(BF) 10111111
(DB) 11011010
03 . BF = DA

B3 = 1011 0011
DA = 1101 1010
69 = 0110 1001

69 = 0110 1001
5D = 0101 1101
34 = 0011 0100

34 = 0011 0100
30 = 0011 0000
04 = 0000 0100

B3 XOR DA XOR 5D XOR 30
69 XOR 5D XOR 30
34 XOR 30
04

Portanto: o primeito byte da primeira coluna da matriz resultante é **04**

MixColumns — Cálculo: Primeiro Elemento da Segunda Coluna

(02 . E0) XOR (03 . B4) XOR (01. 52) XOR (01 . AE) (em FG(2^8))

02 . E0
02 . E0 = 1bit_esq(1110000) = 111000000
1100 0000
(1B) 0001 1011
(DB) 1101 1011
02 . E0 = DB

01 . 52
01 . 52 = 52

03 . B4
(02 . B4) XOR B4
02 . B4 = 1bit_esq(10110100) = 101101000
0110 1000
(1B) 0001 1011
(73) 0111 0011
(B4) 1011 0100
(C7) 1100 0111
03 . B4 = C7

01 . AE
01 . AE = AE

DB = 1101 1011
C7 = 1100 0111
1C = 0001 1100

1C = 0001 1100
52 = 0101 0010
4E = 0100 1110

4E = 0100 1110
AE = 1010 1110
E0 = 1110 0000

DB XOR C7 XOR 52 XOR AE = EO

Portanto: o primeito byte da segunda coluna da matriz resultante é **E0**

0	1	2	3

MixColumns — Misturando Bytes Dentro de Colunas (CORRETO)

1. ESTADO DE ENTRADA
(pós-ShiftRows)

A matriz vem do ShiftRows.
O MixColumns processa cada coluna **INDEPENDENTEMENTE**.

	0	1	2	3
Linha 0	d4	e0	b8	1e
Linha 1	bf	b4	41	27
Linha 2	5d	52	11	98
Linha 3	30	ae	f1	e5

Matriz de Entrada

Fluxo do Algoritmo

O MixColumns processa cada coluna **INDEPENDENTEMENTE**.

2. OPERAÇÃO EM CADA COLUNA
(GF(2^{8}) Multiplicação de Matriz)

Matriz MDS (Constantes)					Coluna de Entrada C_j)		Coluna de Saída (C'_j)
02	03	01	01	X	d4	=	04
01	02	03	01		bf		66
01	01	02	03		5d		81
03	01	01	02		30		e5
(4×4)					(4×1)		(4×1)

- A multiplicação correta é: $C'_j = MDS \times C_j$
- Cada byte da coluna de saída (C'_j) é o XOR de 4 multiplicações em GF(2^8).
- Fórmula para o elemento da linha i:

$$C'_{i,j} = \sum_{k=0}^{3} M_{i,k} \cdot C_{k,j} \quad \text{(em GF}(2^8)\text{)}$$

Multiplicações em GF(2^8) com o polinômio reduzido
$m(x) = x^8 + x^4 + x^3 + x + 1$ (0x11B)

Exemplos úteis:
- **02**·(val) = xtime(val) (desloca à esquerda e aplica 0x**1B** se houver estouro do MSB=1)
- **03**·(val) = **02**·(val) ⊕ (val)
- **01**·(val) = (val)

Exemplo detalhado para a COLUNA 0 ($j = 0$)

$C'_{0,0} = (02 \cdot d4) \oplus (03 \cdot bf) \oplus (01 \cdot 5d) \oplus (01 \cdot 30) = \mathbf{04}$
$C'_{1,0} = (01 \cdot d4) \oplus (02 \cdot bf) \oplus (03 \cdot 5d) \oplus (01 \cdot 30) = \mathbf{66}$
$C'_{2,0} = (01 \cdot d4) \oplus (01 \cdot bf) \oplus (02 \cdot 5d) \oplus (03 \cdot 30) = \mathbf{81}$
$C'_{3,0} = (03 \cdot d4) \oplus (01 \cdot bf) \oplus (01 \cdot 5d) \oplus (02 \cdot 30) = \mathbf{e5}$

O mesmo processo é aplicado às COLUNAS 1, 2 e 3.

3. ESTADO FINAL (Unified)

Cada coluna transformada substitui a coluna original.

C_0 Final	C_1 Final	C_2 Final	C_3 Final	
04	e0	48	28	Linha 0
66	cb	f8	06	Linha 1
81	19	d3	26	Linha 2
e5	9a	7a	4c	Linha 3
Byte 0 now in C_0	Byte 0 now in C_1	Byte 0 now in C_2	Byte 0 now in C_3	

Matriz Resultante

CONSEQUÊNCIA PRINCIPAL:
MixColumns mistura os bytes dentro de cada coluna, espalhando a informação de cada byte e alcançando **DIFUSÃO MÁXIMA.**

PONTO-CHAVE: A ordem correta da multiplicação é: MATRIZ (4×4) x VETOR COLUNA (4×1) = VETOR COLUNA (4×1)

LEGENDA:
- Entrada (pós-ShiftRows)
- Constantes (MDS)
- Saída intermediária (por coluna)
- Resultado final

AddRoundKey — fechando o round 1

Depois do MixColumns, um último XOR com a Round key 1 fecha o round.

Após MixColumns

04	e0	48	28
66	cb	f8	06
81	19	d3	26
e5	9a	7a	4c

⊕

a0	88	23	2a
fa	54	a3	6c
fe	2c	39	76
17	b1	39	05

=

Fim do round 1

a4	68	6b	02
9c	9f	5b	6a
7f	35	ea	50
f2	2b	43	49

As 11 chaves de round, com

Repetindo essa lógica 44 vezes (11 rounds × 4 palavras), chegamos

0	2b7e151628aed2a6abf7158809cf4f3c
1	a0fafe1788542cb123a339392a6c7605
2	f2c295f27a96b9435935807a7359f67f
3	3d80477d4716fe3e1e237e446d7a883b
4	ef44a541a8525b7fb671253bdb0bad00

O QUE ACABAMOS DE COMPLETAR

SubBytes → ShiftRows → MixColumns → AddRoundKey

Todos os rounds intermediários (2 a 9) repetem exatamente esta sequência — só mudam o estado de entrada e a chave de round.

Rounds 2 a 9 — mesma mecânica, novo estado

Nada novo — cada round de 2 a 9 executa exatamente SubBytes → ShiftRows → MixColumns → AddRoundKey, usando sempre a Round key correspondente.

Round	Estado ao final do round (16 bytes)
1	a4686b029c9f5b6a7f35ea50f22b4349
2	aa6182688fddd2325fe34a4603efd29a
3	48674dd66c1de35f4e9db158ee0d38e7
4	e0c8d9859263b1b87f6335bee8c05001
5	f1c17c5d0092c8b56f4c8bd555ef320c
6	263de8fd0e4164d22eb7728b177da925
7	5a19a37a4149e08c42dc1904b11f650c
8	ea04658583455d965c3398b0f02dadc5
9	eb598b1b402ea1c3f23813421e84e7d2

Cada linha depende inteiramente da anterior — é o efeito avalanche se acumulando a cada round.

Round 10 — o round final é diferente

A exceção — o último round PULA o MixColumns — só executa SubBytes → ShiftRows → AddRoundKey.

Por quê — sem essa assimetria, cifrar e decifrar poderiam ser combinados de um jeito que enfraqueceria a segurança; o MixColumns final não agregaria proteção real, só complicaria a decifragem.

```
d014f9a8  c9ee2589  e13f0cc8
b6630ca6
```

Após SubBytes

e9	cb	3d	af
09	31	32	2e
89	07	7d	2c
72	5f	94	b5

Após ShiftRows

e9	cb	3d	af
31	32	2e	09
7d	2c	89	07
b5	72	5f	94

AddRoundKey (RK10)

d0	c9	e1	b6
14	ee	3f	63
f9	25	0c	0c
a8	89	c8	a6

39	02	dc	19
25	dc	11	6a
84	09	85	0b
1d	fb	97	32

```
Estado de entrada do round 10: eb40f21e592ea1848b3813e71bc342d2 (fim do round 9)
```

O texto cifrado final

39	02	dc	19
25	dc	11	6a
84	09	85	0b
1d	fb	97	32

Lendo de volta em ordem de bytes (coluna por coluna):

```
3925841d 02dc09fb dc118597 196a0b32
```

Verificação — esse valor é exatamente o publicado no FIPS-197 (Apêndice B) — confira cifrando o mesmo texto claro com a mesma chave em qualquer biblioteca (OpenSSL, PyCryptodome etc.).

Revertendo o processo — o conceito

A ideia — decifrar não é "rodar para trás" de forma vaga — é aplicar as operações INVERSAS EXATAS, na ORDEM INVERSA, usando as mesmas chaves de round (na ordem reversa: 10, 9, ..., 0).

Etapa da cifragem	Inversa usada na decifragem
`AddRoundKey`	AddRoundKey (é sua própria inversa — XOR desfaz XOR)
`MixColumns`	InvMixColumns — mesma ideia, com a matriz inversa (0e 0b 0d 09)
`ShiftRows`	InvShiftRows — desloca as linhas para a DIREITA
`SubBytes`	InvSubBytes — usa a S-box inversa (tabela diferente)

Detalhe sutil — a ordem de ShiftRows/SubBytes se inverte porque essas operações comutam entre si (uma reorganiza posições, a outra age byte a byte), simplificando a implementação.

Desfazendo o round 10, na prática

Partindo do texto cifrado, primeiro um AddRoundKey com a Round key 10 (XOR é sua própria inversa):

39	02	dc	19
25	dc	11	6a
84	09	85	0b
1d	fb	97	32

⊕ RK10 =

Após AddRoundKey

e9	cb	3d	af
31	32	2e	09
7d	2c	89	07
b5	72	5f	94

Depois, InvShiftRows → InvSubBytes → AddRoundKey (RK9) → InvMixColumns:

```
Apos InvShiftRows:    e9cb3daf 0931322e 89077d2c 725f94b5
Apos InvSubBytes:     eb598b1b 402ea1c3 f2381342 1e84e7d2
Apos AddRoundKey(9): 4740a34c 37d4709f 94e43a42 eda5a6bc
Apos InvMixColumns:   87f24d97 6e4c90ec 46e74ac3 a68cd895
```

Esse último valor já bate exatamente com o "fim do round 8" que tínhamos na cifragem — a prova de que a inversão está correta.

RESULTADO

De volta ao texto claro original

Repetindo o processo — InvShiftRows → InvSubBytes → AddRoundKey(RK9) → InvMixColumns por mais 8 rounds (repare que cada resultado intermediário bate, byte a byte, com o estado equivalente calculado na cifragem).

Round final da decifragem — o último round a desfazer (round 1 original) não leva InvMixColumns — assim como o round 10 não levou MixColumns na cifragem.

32	88	31	e0
43	5a	31	37
f6	30	98	07
a8	8d	a2	34

3243f6a8 885a308d 313198a2 e0370734

idêntico ao texto claro do início — o ciclo fechou.

Do zero (parte 1 de 5): aritmética e chaves

Sem bibliotecas externas — só para expor exatamente o que uma lib de criptografia faz por dentro.

```
def gmul(a, b):
    # multiplicacao de bytes em GF(2^8)
    p = 0
    for _ in range(8):
        if b & 1: p ^= a
        carry = a & 0x80
        a = (a << 1) & 0xFF
        if carry: a ^= 0x1B
        b >>= 1
    return p
def key_expansion(key, Nk=4, Nr=10):
    w = [list(key[4*i:4*i+4]) for i in range(Nk)]
    for i in range(Nk, 4*(Nr+1)):
        temp = w[i-1][:]
        if i % Nk == 0:
            temp = sub_word(rot_word(temp))
            temp[0] ^= RCON[i//Nk - 1]
        w.append([w[i-Nk][j]^temp[j] for j in range(4)])
    return w
```

Do zero (parte 2 de 5): estado e transformações

```
def bytes_to_state(b):
    s = [[0]*4 for _ in range(4)]
    for c in range(4):
        for r in range(4):
            s[r][c] = b[4*c + r]   # preenche por coluna
    return s

def sub_bytes(state):
    return [[SBOX[state[r][c]] for c in range(4)]
            for r in range(4)]

def shift_rows(state):
    return [state[r][r:] + state[r][:r]
            for r in range(4)]

def add_round_key(state, round_words):
    rk = state_to_bytes([[round_words[c][r]
         for c in range(4)] for r in range(4)])
    flat = state_to_bytes(state)
    return bytes_to_state([flat[i]^rk[i]
                           for i in range(16)])
```

Do zero (parte 3 de 5): MixColumns e o loop de rounds

```
MIX_MATRIX = [[2,3,1,1],[1,2,3,1],
              [1,1,2,3],[3,1,1,2]]

def mix_columns(state):
    new = [[0]*4 for _ in range(4)]
    for c in range(4):
        col = [state[r][c] for r in range(4)]
        mc = _mix_col(col, MIX_MATRIX)
        for r in range(4):
            new[r][c] = mc[r]
    return new

def aes128_encrypt_block(plaintext, key):
    w = key_expansion(list(key))
    state = bytes_to_state(list(plaintext))
    state = add_round_key(state, w[0:4])
    for rnd in range(1, 11):
        state = sub_bytes(state)
        state = shift_rows(state)
        if rnd != 10:
            state = mix_columns(state)
        state = add_round_key(state, w[4*rnd:4*rnd+4])
    return bytes(state_to_bytes(state))
```

Do zero (parte 4 de 5): a função de decifrar

```
def aes128_decrypt_block(ciphertext, key):
    w = key_expansion(list(key))
    state = bytes_to_state(list(ciphertext))
    state = add_round_key(state, w[40:44])

    for rnd in range(10, 0, -1):
        state = inv_shift_rows(state)
        state = inv_sub_bytes(state)
        rk = w[4*(rnd-1):4*(rnd-1)+4]
        state = add_round_key(state, rk)
        if rnd != 1:
            state = inv_mix_columns(state)

    return bytes(state_to_bytes(state))
```

Note a simetria — é literalmente o mesmo laço da cifragem, só com as operações trocadas pelas inversas e o range andando de 10 até 1.

Do zero (parte 5 de 5): rodando com o vetor oficial

```
plaintext = bytes.fromhex('3243f6a8885a308d313198a2e0370734')
key = bytes.fromhex('2b7e151628aed2a6abf7158809cf4f3c')

c = aes128_encrypt_block(plaintext, key)
p = aes128_decrypt_block(c, key)
```

SAÍDA NO TERMINAL

```
Texto cifrado:    3925841d02dc09fbdc118597196a0b32
(confere com o vetor de teste oficial do FIPS-197)

Texto recuperado: 3243f6a8885a308d313198a2e0370734
(recuperou exatamente o texto claro original)
```

Usando as bibliotecas Python: cifrar e decifrar

```python
import os
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
# ============================================================================
# 1. Geração da chave
# ============================================================================
# AES-256: chave de 32 bytes. Pode ser gerada aleatoriamente ou derivada de uma senha (com PBKDF2/Argon2). Aqui geramos aleatoriamente.
key = AESGCM.generate_key(bit_length=256)
aesgcm = AESGCM(key)

# ============================================================================
# 2. Cifrar a mensagem
# ============================================================================
# plaintext = mensagem original (bytes)
# associated_data = dados autenticados mas NÃO cifrados (opcional)
# nonce = número único por operação (NUNCA reutilizar com a mesma chave!)
plaintext = b"Minha mensagem secreta para a aula de criptografia"
nonce = os.urandom(12)    # 96 bits, recomendado para GCM
associated_data = b"aula-crypto-2026"

ciphertext = aesgcm.encrypt(nonce, plaintext, associated_data)

print(f"Mensagem original: {plaintext.decode()}")
print(f"Cifrado (hex): {ciphertext.hex()}")
print(f"Nonce (hex): {nonce.hex()}")
```

O modo GCM é a escolha moderna: além de cifrar, ele autentica a mensagem, detectando qualquer adulteração.

Decifrar

```python
# ================================================
# 3. Decifrar a mensagem (com a MESMA chave e o MESMO nonce)
# ================================================
plaintext_recuperado = aesgcm.decrypt(nonce, ciphertext, associated_data)

print(f"Mensagem decifrada: {plaintext_recuperado.decode()}")
print(f"Igual à original? {plaintext_recuperado == plaintext}")
```

Saída típica

```
Mensagem original: Minha mensagem secreta para a aula de criptografia
Cifrado (hex): 8f3a2b... (binário)
Nonce (hex): a1b2c3d4e5f6...
Mensagem decifrada: Minha mensagem secreta para a aula de criptografia
Igual à original? True
```

Links interessantes

- https://colab.research.google.com/drive/1d_KMCCoHcyvJFnF2FL8-7nUDJuMFTxCH#scrollTo=y-QJPFSuCcG1
- https://cryptohack.org/courses/symmetric/course_details/
- https://cryptopals.com/
- https://www.garykessler.net/library/crypto.html
- https://www.schneier.com/

RESUMO

O que vimos, do início ao avançado

1. O texto claro vira uma matriz 4×4 (state), preenchida coluna por coluna
2. O key schedule expande 16 bytes em 11 chaves de round, usando a S-box e Rcon
3. Cada round mistura SubBytes (não linear), ShiftRows e MixColumns (difusão via GF(2^8)) e AddRoundKey
4. O último round pula o MixColumns — uma assimetria proposital do design
5. Decifrar aplica as inversas exatas, na ordem inversa, com as mesmas chaves de round
6. Todo o algoritmo cabe em ~150 linhas de Python sem nenhuma biblioteca externa

Obrigado(a)!

Referências

- STALLINGS, W.; BROWN, L. Computer Security: Principles and Practice. 5th ed. Pearson, 2023. Capítulos sobre criptografia simétrica.
- NIST FIPS 197 — Advanced Encryption Standard (AES). National Institute of Standards and Technology.
- NIST SP 800-38A/D — Recommendation for Block Cipher Modes of Operation (incluindo GCM).
- Documentação oficial do pacote Python cryptography (pyca/cryptography).
- ANDERSON, R. Security Engineering. 3rd ed. Wiley — capítulo sobre criptografia aplicada.